TATA TECHNOLOGIES LTD.

TECH PULSE FY-26 | AI/ML Capstone Project

PROJECT SYNOPSIS

Student Details

Institute/University: Pimpri Chinchwad College of Engineering (PCCOE), Pune
Programme / Batch: B.Tech Information Technology / Batch 2023–2027
Student Name: Tanmay Shah
PRN: 123B1F096

1. Case Study ID and Project Title / Topic

Case Study ID (CS1 to CS5): CS4
Project Title / Topic: AutoCodeGuard — An Automotive Secure Code Review Assistant

2. Objectives

Define the specific objectives and focus areas of the allocated AIML case study.

Objectives

- **Automated Static Defect Detection:** Implement a deterministic static analysis engine leveraging Python Abstract Syntax Trees (AST) and regex heuristics to flag high-severity programming defects (arbitrary code execution via eval/exec, shell injection in subprocess, hardcoded credentials, weak hashing, and unsafe error handling).
- **Advisory LLM-Powered Code Review:** Integrate Google Gemini (gemini-flash-lite-latest) through the google-genai SDK to provide advisory contextual analysis, explain security ramifications, evaluate risk levels, and suggest actionable remediation without executing user code.
- **Contextual Guideline Retrieval:** Deploy an in-memory TF-IDF vectorization and cosine-similarity retrieval system over a local JSON automotive knowledge base (referencing MISRA C:2012, ISO 26262 Part 6, ISO/SAE 21434, and OWASP).
- **Secure Findings Dashboard & Triage:** Deliver an intuitive Streamlit interface featuring dual input modes (paste and multi-format upload), severity distribution metrics, finding expanders, and structured report exports (JSON and CSV).
- **Measurable & Reproducible Evaluation:** Validate the deterministic analyzer against a 12-case benchmark suite to empirically calculate Precision, Recall, and F1 scores while keeping heuristic metrics separate from generative LLM outputs.

3. Scope

Clearly state what will be implemented in the project and what will not be covered. Respect mandatory controls such as citations, human review, and approved or synthetic data.

In Scope – To Be Implemented

- **Deterministic Static Rules:** 6 Python AST rules (eval/exec, subprocess shell=True, bare except, assert validation, mutable defaults, os.system) and 6 language-agnostic regex rules (credentials, MD5/SHA1, SQL injection formatting, TODO/FIXME markers, DEBUG=True, print diagnostics).
- **Gemini API Integration:** Read-only advisory review using gemini-flash-lite-latest with structured JSON response mode, fallback cascade across Lite models, and approximate line-number sanity verification.
- **Local Knowledge Base & Retrieval:** TF-IDF matching over 16 curated guidelines spanning General, MISRA-inspired, Automotive, and Security standards.
- **Streamlit User Interface:** Interactive code input, privacy consent notification, real-time metrics cards, and report download.

- **Empirical Benchmark Evaluation:** Automated evaluation script (evaluate.py) computing TP, FP, FN, precision, recall, and F1 on a 12-case dataset.

Out of Scope – Will Not Be Implemented

- **Commercial SAST Certification:** The tool is an educational prototype and does not substitute for certified industrial tools (Coverity, Polyspace) or full MISRA compliance certification.
- **Autonomous Code Execution / Patching:** User-submitted code is never executed, compiled, or modified autonomously.
- **Persistent Data Storage / Database:** No persistent database server or cloud telemetry; code is processed strictly in memory.
- **Heavyweight Model Training:** No fine-tuning or local deployment of massive neural models; no heavy vector databases (ChromaDB/FAISS) requiring dedicated GPU infrastructure.

4. Proposed Approach

Pre-trained model (name, version, local/private hosting)

Google Gemini (gemini-flash-lite-latest, google-genai SDK v2.30.1), accessed remotely via Google Generative AI REST API with structured JSON output enforcement (temperature=0.2). Automated fallback cascade includes gemini-3.5-flash-lite, gemini-3.1-flash-lite, and gemini-3.8-flash.

Embedding model

Scikit-Learn TfidfVectorizer (unigram and bigram representation with English stop-words removal). No external embedding API or large neural embedding models are downloaded.

Vector store (FAISS / ChromaDB)

In-memory sparse TF-IDF matrix indexed over guidelines_kb.json, paired with cosine_similarity from scikit-learn. Heavy vector stores (FAISS / ChromaDB) are intentionally omitted to maintain a lightweight, zero-database architecture.

RAG pipeline and/or AI-agent design – brief description

Hybrid two-phase pipeline: (1) Deterministic AST & Regex scans flag syntactic evidence with line numbers. (2) Each finding's text queries the TF-IDF index to retrieve top-k (default k=2) relevant standards. (3) Upon explicit user request and privacy consent, source code is dispatched to Gemini with a read-only security persona. Gemini's structured JSON findings are validated, line-checked against source boundaries, and enriched with retrieved guideline references.

Service layer and UI (e.g., FastAPI / Flask / Streamlit)

Streamlit web application (app.py) featuring configuration sidebars, dual input tabs (paste and multi-format upload), metric summary cards, expandable findings displays, and JSON/CSV export buttons.

Tools / deterministic rules, if applicable

12 deterministic rules across AST (ACG-PY-001 to ACG-PY-006) and Regex (ACG-RE-001 to ACG-RE-006) executing safely in memory without invoking external interpreters.

Proposed System / Architecture

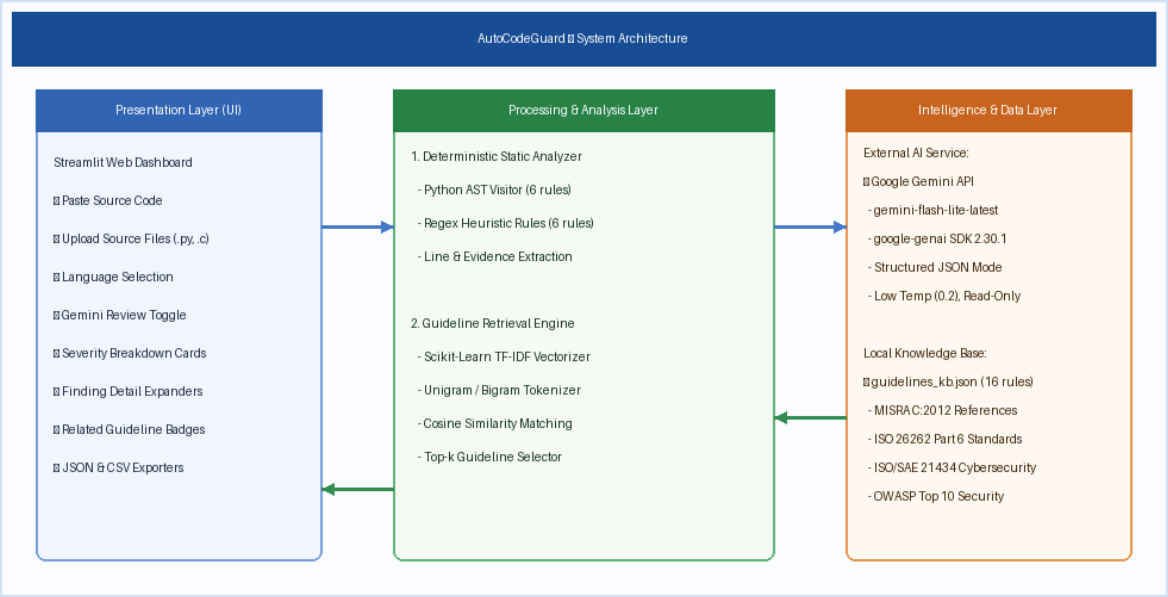


Figure 1: AutoCodeGuard Proposed System Architecture

5. Input Data / Knowledge Base

Use approved or synthetic data only.

Data / Knowledge Sources

• Automotive Coding Guidelines Knowledge Base (guidelines_kb.json): 16 curated guidelines referencing MISRA C:2012, ISO 26262 Part 6, ISO/SAE 21434, and OWASP.
• Synthetic Benchmark Dataset (evaluation_dataset.json): 12 synthetic test cases containing known flaws and safe control implementations.
• Demonstration Test Files (sample_vulnerable_ecu.py, sample_safe_telematics.py).

Approximate Size and Format

• guidelines_kb.json: 16 entries, 6.9 KB, JSON format.
• evaluation_dataset.json: 12 test cases, 4.3 KB, JSON format.
• Sample ECU code files: 2 files, ~3.4 KB total, UTF-8 text.

Preprocessing and EDA Planned

Text normalization, stop-word filtering, and n-gram tokenization for guideline corpus. AST grammar parsing and regex compilation for code verification. JSON schema validation and boundary checks on LLM responses.

6. Expected Outcomes and Evaluation Plan

Expected Outputs – aligned to the case study

Real-time review dashboard displaying severity breakdown, detected issues, approximate line references, code evidence, explanations, and remediation advice. Downloadable structured JSON reports and CSV summaries.

Evaluation Method – test question set / ground truth / metrics / validation approach

Deterministic analyzer evaluated against the 12-case benchmark dataset via evaluate.py. Metrics measured: True Positives (TP), False Positives (FP), False Negatives (FN), Precision, Recall, and F1 score. Separate qualitative verification of Gemini advisory review schema validity and hallucination rate.

Responsible AI Measures – grounding and citations / privacy / security / human oversight

• Grounding & Citations: Heuristic findings tied directly to source line evidence; LLM findings augmented with retrieved guideline citations.

• Privacy: Explicit privacy notice before sending code; API key loaded from .env and never logged or committed.
• Security: Zero code execution; read-only reviewer persona.
• Human Oversight: All findings designated as potential issues requiring human engineering review.

MLOps / Deployment Elements Planned – versioning / Docker / tracking

Git version control hosted on GitHub (https://github.com/tanmayshah2705/tata_techpulse_prj). Pinned dependencies in requirements.txt. Model configuration tracked via model_config.json.

7. Planned AI-Tool Usage

Mention the AI tools / models used for coding assistance and specify the purpose and project artifact affected. This information will be carried into the final AI-tool usage declaration.

AI Tool / Model

Google Antigravity (AI Coding Assistant) & Google Gemini API (gemini-flash-lite-latest)

Purpose

Scaffolding project structure, generating unit tests, assisting in documentation drafting, and providing runtime advisory code review.

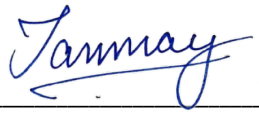
Artifact Affected

Code (app.py, static_analyzer.py, gemini_integration.py), Documentation (Synopsis, Technical Report), Evaluation harness (evaluate.py).

Student Declaration

[X] I will not change the approved case-study scope, major methodology, or implementation approach without prior faculty approval.

[X] I confirm that the work submitted under this synopsis represents my own understanding and contribution, and I will be responsible for explaining every significant part of the selected case study and its implementation.

Name and Signature:  Date: 14/10/2026

Tanmay Shah